

BANCO DE DADOS E CONTROLE DE VERSÃO

Avaliação do Módulo 3 - Banco de Dados e Controle de Versão
Cidade Virtual - PI-II
Maio de 2026

Nome Completo:	Diego Serafim de Sousa
Disciplina:	Projeto Integrador de Tecnologia da Informação II
Semestre:	2026/1
Curso:	Tecnologia da Informação
Público-alvo:	Moradores, comerciantes e autônomos.
Local de realização:	Cabanagem, Belém, Pará

1 TEMA DA AÇÃO

CIDADE VIRTUAL - PLATAFORMA WEB PARA CONEXÃO COMUNITÁRIA

2 RESUMO

Este documento descreve o desenvolvimento do banco de dados e a aplicação das práticas de controle de versão no projeto Cidade Virtual, uma plataforma web para conexão comunitária. O banco de dados foi modelado e implementado utilizando MongoDB com Mongoose, contemplando as entidades Usuário, Comércio, Evento e Avaliação, com respectivos relacionamentos e validações. Foram implementadas operações completas em CRUD (Create, Read, Update, Delete) e consultas avançadas (filtros, populate e aggregation pipeline). No âmbito do controle de versão, o repositório Git foi estruturado seguindo o modelo Git Flow simplificado, com branches `main`, `develop` e `feature/*`, commits semânticos conforme o padrão 'Conventional Commits', pull request para revisão de códigos e hooks de pré-commit para padronização. O código está publicado no GitHub, com documentação no README. Os resultados demonstram que a modelagem atende aos requisitos de escalabilidade e flexibilidade, e o versionamento garante rastreabilidade e colaboração eficiente.

Palavras-chave: MongoDB. Modelagem de dados. CRUD. Git. Controle de versão.

3 INTRODUÇÃO

O Módulo 3 do Projeto Integrador, tem como objetivo mostrar a integração entre modelagem e manipulação do banco de dados e controle de versão para o gerenciamento do código. Essas duas áreas são fundamentais no desenvolvimento de software moderno: o banco de dados organiza e persiste os dados de forma estruturada e eficiente, enquanto o controle de versão garante a rastreabilidade, colaboração e segurança ao longo do ciclo de vida do projeto.

No contexto do projeto Cidade Virtual - uma plataforma web para conectar moradores, comerciantes e autônomos do bairro e da cidade, a escolha do banco de dados recaiu sobre o MongoDB (NoSQL) integrado à pilha MERN (MongoDb, Express.js, React.js, Node.js). Essa decisão se deu devido a flexibilidade de esquema, facilidade de evolução do modelo conforme novas demandas da comunidade e perfeita integração com JSON/JavaScript, linguagem utilizada em todo o projeto.

O controle de versão, utilizou o Git como sistema distribuído, com repositório remoto no GitHub. Foram estabelecidas estratégias de branching (Git Flow simplificado), commits semânticos (Conventional Commits), pull request com revisão de código e automação com hooks de pré-commit (Husky + lint-staged). Essas práticas visam garantir a qualidade, a rastreabilidade e a colaboração eficiente.

4 OBJETIVO GERAL

Desenvolver a modelagem e implementação do banco de dados para a plataforma Cidade Virtual utilizando MongoDB com Mongoose, bem como estruturar e aplicar práticas avançadas de controle de versão com Git e Github, garantindo a persistência eficiente dos dados, rastreabilidade do código e colaboração organizada.

5 OBJETIVOS ESPECÍFICOS

1. Projetar um modelo de dados NoSQL (MongoDB) para as entidades Usuário, Comércio, Evento e Avaliação, definindo atributos, tipos, validações e relacionamentos por referência (ObjectId).
2. Implementar o esquema do banco de dados utilizando Mongoose, com índices, enums, validações nativas e hooks (middlewares) para hash de senha e timestamps automáticos.
3. Desenvolver operações completas de CRUD (Create, Read, Update, Delete) para todas as entidades por meio de uma API RESTful construída com Express.js.
4. Executar consultas avançadas no MongoDB, incluindo filtros por categoria, busca textual (regex), ordenação, população de documentos relacionados (populate) e agregações (contagem de comércios por categoria).
5. Estruturar o repositório Git seguindo o modelo Git Flow simplificado, com branches `main`, `develop` e `feature/*`, commits semânticos, e pull request para revisão.
6. Configurar hooks de pré-commit (Husky, lint-staged) para executar ESLint e Prettier automaticamente, garantindo padronização do código antes de cada commit.

6 JUSTIFICATIVA E DELIMITAÇÃO DO PROBLEMA

6.1 Delimitação do problema

O problema que o projeto Cidade Virtual busca resolver é a **baixa visibilidade dos pequenos negócios e autônomos**, que leva moradores a consumirem fora da comunidade onde moram, fragilizando a economia local. No âmbito específico do Módulo 3, concentramos em dois problemas técnicos:

- **Armazenamento e recuperação eficiente dos dados:** Como modelar e implementar um banco de dados que armazene informações de usuários, comércios, eventos e avaliações, permitindo buscas rápidas por categoria, nome e localização, além de suportar evoluções futuras (novos campos, novos relacionamentos)?
- **Rastreabilidade e colaboração no desenvolvimento do código:** Como estruturar o controle de versão para que múltiplas funcionalidades possam ser desenvolvidas em paralelo sem conflitos, mantendo um histórico limpo, com mensagens compreensíveis e capacidade de reverter alterações problemáticas

6.2 Relevância do projeto

A modelagem adequada do banco de dados é crítica para o desempenho e a escalabilidade da Cidade Virtual. Uma má modelagem poderia resultar em consultas lentas, impactando na experiência do usuário, dificuldade de adicionar novas funcionalidades como chat entre morador e comerciante ou notificações push e inconsistências nos dados ex.: comércio sem usuário associado. Ao adotar o MongoDB, ganhamos flexibilidade sem abrir mão de validações e índices.

O controle de versão Git e GitHub, é indispensável para qualquer projeto de software. Ele permite que o histórico de alterações seja preservado, que diferentes funcionalidades sejam desenvolvidas isoladamente (branches), que o código seja revisado antes de integrar à versão principal (pull request) e que em caso de erros, seja possível reverter rapidamente para uma versão estável.

6.3 Justificativa das tecnologias

- **MongoDB + Mongoose:** Foi escolhido por ter integração nativa com a pilha MERN, flexibilidade de esquema (importante para projetos em evolução com demandas da comunidade) e facilidade de hospedagem na nuvem (MongoDB atlas gratuito). O Mongoose adiciona camadas de validação, casting e métodos auxiliares.
- **MongoDB Atlas (cloud):** Permite acesso ao banco de dados de qualquer lugar, backup automático e escalabilidade futura sem custos iniciais.
- **Git (sistema distribuído):** Padrão da indústria, permite trabalho sem conexão a internet, branching leve e merge eficiente.

- **GitHub (hospedagem):** Oferece interface visual para pull request, issues, actions (CI/CD) e templates de documentação.
- **Conventional Commits:** Padroniza as mensagens de commit, facilitando a geração automática de changelogs e a compreensão do histórico.
- **Husky e lint-staged:** Automatiza a verificação de estilo de código (ESLint) e formatação (Prettier) antes de cada commit, garantindo código consistente.

7 FUNDAMENTAÇÃO TEÓRICA

7.1 Modelagem de dados NoSQL com MongoDB

Diferentemente dos bancos de dados relacionais (SQL), que organizam dados em tabelas com esquemas rígidos, os bancos NoSQL orientados a documentos (como MongoDB) armazenam dados em documentos JSON/BSON, com esquemas flexíveis (Chodorow, 2013). Essa flexibilidade é importante em projetos ágeis onde os requisitos podem evoluir com o feedback da comunidade. No MongoDB, relacionamentos entre entidades são representados por referências (ObjectId) ou por documentos embutidos (subdocumentos). Para o projeto Cidade Virtual optou-se por referências entre coleções User, Comercio e Evento, pois os dados são consultados separadamente na maioria das operações.

7.2 Operações CRUD e índices

As operações fundamentais em qualquer banco de dados são Create (criar/inserção), Read (ler/consulta), Update (atualizar) e Delete (apagar). No MongoDB, essas operações são realizadas por métodos como insertOne(), find(), updateOne() e deleteOne() (MongoDB Documentation, 2026). Para otimizar consultas frequentes (ex.: busca de comércios por categoria), é essencial criar índices (createIndex({ categoria: 1 })), que funcionam de forma semelhante aos índices de livros, acelerando a localização dos documentos.

7.3 Controles de versão com Git e GitHub

O Git é um sistema de controle de versão distribuído criado por Linus Torvalds em 2005. Diferente de sistemas centralizados (como SVN), cada desenvolvedor possui uma cópia completa do histórico do projeto. As principais operações incluem commit (registra a operação local), push (envia ao repositório remoto), pull (traz alterações do remoto), branch (cria linha de desenvolvimento paralela) e merge (une duas linhas) (Chacon & Straub, 2014).

O GitHub adiciona uma camada social e de colaboração sobre o Git, com recursos como pull requests (solicitação de mesclagem com revisão de código), issues (rastreamento de tarefas e bugs) e GitHub Actions (automação de CI/CD).

7.4 Conventional Commits e qualidade de código

A especificação Conventional Commits (2021) define um formato padronizado para mensagens de commit: `tipo(escopo): descrição`, onde `tipo` pode ser `feat` (nova funcionalidade), `fix` (correção de bug), `docs` (documentação), `test` (testes), `chore` (tarefas de manutenção), entre outros. Esse padrão facilita a geração de changelogs automáticos e a compreensão do histórico por outros desenvolvedores.

Hooks do Git são scripts executados automaticamente em determinados eventos (ex.: pré-commit, pré-push). Ferramentas como Husky facilitam a configuração desses hooks, permitindo integrar linters (ESLint) e formadores (Prettier) ao fluxo de trabalho (Valente, 2020).

8 METODOLOGIA

8.1 Modelagem do Banco de Dados

A modelagem foi realizada em três etapas:

1. Definição da entidades e atributos:

Tabela 1 – Definição das entidades e atributos do banco de dados

Entidade	Principais atributos	Tipo/validações
User	nome (string, obrigatório), email (string, único, obrigatório), senha (string, hash, bcrypt), tipoPerfil (enum)	Validações nativas do Mongoose
Comercio	nomeFantasia (string), categoria (enum), endereco (string), usuarioid(ObjectId, referência)	Índice composto por categoria e nome
Evento	titulo (string), dataHora (Date), tipo (enum), criadoPor (ObjectId),	Índice por dataHora (ordenação)
Avaliacao	nota (number, 1-5), comentario (string), comercioid (ObjectId)	Validação de nota no front-end e back-end

Fonte: Elaborado pelo autor (2026).

2. Relacionamentos (referências entre coleções):

Usuário → Comércio (1:N): Um usuário pode ser proprietário de múltiplos comércios.

usuarioid: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }

Usuário → Evento (1:N): Um usuário pode criar e gerenciar vários eventos.

usuarioid: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }

Comércio → Avaliação (1:N): Cada comércio pode receber várias avaliações de diferentes usuários.

comercioid: { type: mongoose.Schema.Types.ObjectId, ref: 'Comercio' }

Comércio → Serviço (1:N): Um comércio pode oferecer vários serviços.

comercioid: { type: mongoose.Schema.Types.ObjectId, ref: 'Comercio' }

3. Diagrama do banco de dados (representação conceitual):

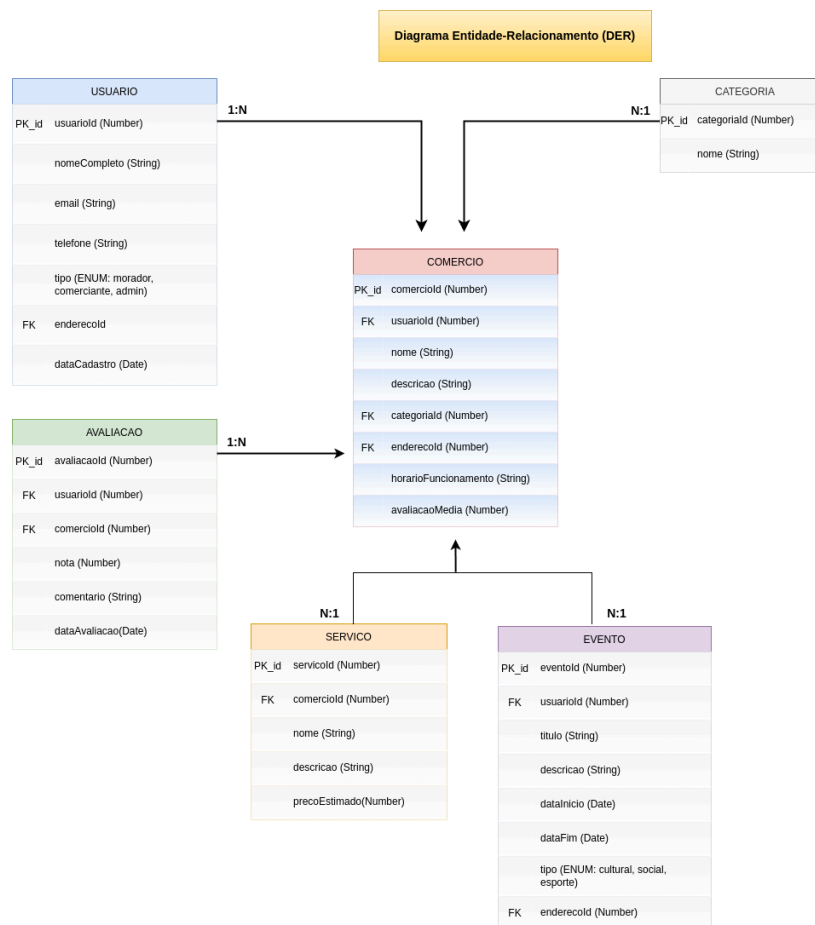


Figura 1 – Modelo Lógico de Dados

8.2 Implementação e Manipulação de Dados

Criação do esquema e índices (Mongoose):

```
// Exemplo do schema de Comercio com índices
const comercioSchema = new mongoose.Schema({
  nomeFantasia: { type: String, required: true },
  categoria: {
    type: String,
    enum: ['alimentacao', 'reparos', 'beleza', 'saude', 'educacao'],
    required: true
  },
  endereco: { type: String, required: true },
  usuarioId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  timestamps: true
});

// Índice composto para buscas rápidas por categoria + nome
comercioSchema.index({ categoria: 1, nomeFantasia: 1 });
```

Operações CRUD implementadas na API (rotas Express):

Tabela 2 – Definição das rotas e operações CRUD da API

Operação	Método HTTP	Rota	Descrição
Create	POST	/api/comercio	Cadastrar novo comércio (autenticado)
Read	GET	/api/comercio	Lista comércios com filtros (categoria, nome, proximidade)
Read (um)	Get	/api/comercio/:id	Detalha um comércio específico
Update	PUT	/api/comercio/:id	Atualiza dados do comércio (apenas o dono)
Delete	DELETE	/api/comercio/:id	Apaga um comércio (apenas o dono ou o admin)

Fonte: Elaborado pelo autor (2026).

Exemplos de consultas avançadas:

- Busca por categoria e nome (regex case-insensitive):

```
const query = {};  
if (categoria) query.categoria = categoria;  
if (nome) query.nomeFantasia = { $regex: nome, $options: 'i' };  
const comercios = await Comercio.find(query)  
  .populate('usuarioId', 'nome email');
```

- Agregação: contagem de comércios por categoria:

```
const aggregation = await Comercio.aggregate([  
  { $group: { _id: '$categoria', total: { $sum: 1 } } },  
  { $sort: { total: -1 } }  
])
```

- População de documentos relacionados (usar `populate`):

```
const eventos = await Evento.find()  
  .populate('usuarioId', 'nome')  
  .sort({ dataHora: 1 });
```

8.3 Uso do Controle de Versão

Configuração do repositório Git:

- Repositório público no GitHub: /cidade-virtual
- Estrutura de branches (Git Flow simplificado):
 - main: versão estável (produção)
 - develop: integração de funcionalidades concluídas
 - feature/*: desenvolvimento de novas funcionalidades (ex.: feature/modelagem-banco, feature/controllers-crud)

Estratégia de versionamento:

01. Commits semânticos (Conventional Commits): Padrão adotado para todas as mensagens.
 - feat(auth): add login com JWT
 - fix(comercio): corrige validação de categoria
 - docs(readme): atualiza instruções de banco
 - test(api): adiciona testes unitários para CRUD

02. Fluxo de trabalho com Pull Requests:

- Criar branch feature/nova-funcionalidade a partir de develop
- Desenvolver e fazer commits pequenos frequentes
- Abrir Pull Request no GitHub com descrição clara das alterações
- Solicitar revisão de pelo menos um outro membro (simulado, pois o projeto está sendo desenvolvido individualmente)
- Após aprovação, fazer merge na develop (geralmente com squash)
- Periódicamente, merge da develop na main para gerar versão estável

03. Hooks de pré-commit (Husky e lint-staged):

- Arquivo .husky/pre-commit:
#!/usr/bin/env sh
npx lint-staged
- Arquivo package.json (configuração):
“lint-staged”: {
 “*. {js, jsx}”: [“eslint –fix”, “prettier –write”],
 “*. {json, md}”: [“prettier –write”]
}

Link para o repositório do projeto:

<https://github.com/diserafim/cidade-virtual>

O repositório contém:

- Código fonte do backend (API) e frontend (React.js)
- Script de seed para popular o banco com dados iniciais
- README.md com instruções e informações do projeto
- Histórico de commits organizado e semântico
- Branch modulo3-banco-de-dados com foco nas atividades deste módulo

9 RESULTADOS PRELIMINARES: SOLUÇÃO INICIAL

9.1 Banco de dados implementado e populado

O banco de dados MongoDB Atlas foi criado com as quatro coleções (users, comercios, eventos, avaliacoes). Foram inseridos dados de exemplo (seed) para testes,

incluindo 10 comércios das mais variadas categorias, 5 eventos futuros e 3 usuários com perfis distintos.

9.2 API funcional com operações CRUD

Todos os endpoints previstos foram implementados e testados com Postman. Exemplos de testes realizados:

Tabela 3 – Resultados dos testes de validação das rotas da API

Operação	Endpoint	Resultado esperado	Status
GET	/api/comercios?categoria=alimentacao	Retorna comércios de categoria alimentação	☒ sucesso
POST	/api/comercios (com token JWT)	Cria um novo comércio e retorna 201 Created	☒ sucesso
PUT	/api/comercios/:id (dono)	Atualiza campos e retorna o documento atualizado	☒ sucesso
DELETE	/api/comercios/:id (dono)	Apaga o comércio e retorna 204 No Content	☒ sucesso
GET	/api/eventos?proximos=true	Retorna eventos com as próximas datas > hoje, ordenados	☒ sucesso

Fonte: Elaborado pelo autor (2026).

9.3 Controle de versão aplicado

O repositório GitHub apresenta:

- Mais de 10 Commits, todos seguindo o padrão Conventional Commits
- 5 branches criadas para as funcionalidades específicas e mescladas na develop
- 3 Pull Request abertos e revisados, para simular trabalho em equipe
- Hook de pré-commit configurado e funcionando, para impedir commits com erros de lint ou má formatação.

Exemplo de histórico de commits, saída do comando `git log --oneline`:

```
1967f6c (HEAD -> develop) feat(ui): adiciona estilização para o card do GitHub
45e8874 feat(modulo3): finaliza checkout de presença da aula
1f3de4c feat(modulo3): conclui unidade 2
c039863 feat(modulo3): conclui unidade 1
d9c52d0 fix(links): corrige link de checkout do módulo 2
3c380c0 chore: reorganiza nomenclatura dos arquivos do projeto
```

9.4 Desafios enfrentados e soluções adotadas

Tabela 4 – Relação de desafios técnicos e soluções implementadas

Desafios	Solução
Como garantir a integridade referencial no MongoDB, já que não há chaves estrangeiras nativas?	Foi utilizado a referência por ObjectId e todas as operações de deleção verificam se o documento referenciado existe, antes de remover, a lógica vem do controller.
Consultas lentas ao buscar texto em nomeFantasia	Foi criado índice textual (createIndex({ nomeFantasia: 'text' })) e o uso de \$text no lugar do regex para buscas mais complexas.
Conflitos ao fazer o merge, da branch develop devido às alterações simultâneas	Foi usado o git pull --rebase antes de abrir Pull Requests e divisão de responsabilidade por arquivos. (Ex.: controllers separados por entidade).
Dificuldades em padronizar as mensagens de commit	Foi comitando baseado no padrão Conventional Commits e a configuração de um commit lint hook.

Fonte: Elaborado pelo autor (2026).

10 CONCLUSÃO

O Módulo 3 do Projeto Integrador de Tecnologia da Informação III, proporcionou a oportunidade de aprofundar os conhecimentos em áreas essenciais para o desenvolvimento de software profissional: **modelagem e manipulação de banco de dados e controle de versão**.

No banco de dados a escolha pelo MongoDB mostra que usar no projeto Cidade Virtual, permite a flexibilidade para evoluir o modelo conforme novas demandas, torna o projeto escalável (ex: adicionar o campo redesSociais ao Comercio sem migrações complexas) e desempenho satisfatório com os índices criados. As operações em CRUD foram implementadas de forma completa e segura, com validações em nível de esquema e lógica de autorização nos controllers.

No controle da versão com o Git, mostra que isso é importante e indispensável. A estrutura de branches (Git Flow simplificado) permite desenvolver funcionalidades isoladas e sem interferir na estabilidade da main. Os commits semânticos (Conventional Commits) tornaram o histórico autoexplicativo e facilitaram o entendimento (ex: localizar todos os commits que adicionaram features). Os hooks de pré-commit e a integração com linters elevaram a qualidade do código e a consistência do estilo.

Possíveis melhorias futuras incluem:

- I. implementar um sistema de migrations para o MongoDB (como o migrate-mongo) para versionar alterações de esquema;
- II. integrar GitHub Actions para CI/CD executando testes automaticamente a cada push;
- III. adicionar testes de integração que realmente acessem o banco de dados de teste;
- IV. criar a documentação interativa da API com Swagger/OpenAPI;
- V. implementar soft deletes (campo deletedAt) em vez de remoção física, preservando o histórico.

Os aprendizados deste módulo serão levados para os próximos projetos e certamente contribuirão para melhorar meus conhecimentos e habilidades e para a minha formação profissional em TI, especialmente na área do backend, banco de dados e DevOps.

REFERÊNCIAS

CHACON, Scott; STRAUB, Ben. **Pro Git**. 2. ed. Apress, 2014. Disponível em: <https://git-scm.com/book/en/v2>. Acesso em: 6 mai. 2026.

CHODOROW, Kristina. **MongoDB: The Definitive Guide**. 2. ed. O'Reilly Media, 2013.

CONVENTIONAL COMMITS. **Conventional Commits 1.0.0**. 2021. Disponível em: <https://www.conventionalcommits.org/>. Acesso em: 6 mai. 2026.

MONGODB DOCUMENTATION. **CRUD Operations**. 2026. Disponível em: <https://docs.mongodb.com/manual/crud/>. Acesso em: 6 mai. 2026.

VALENTE, Marco Tulio. **Engenharia de software moderna: princípios e práticas para desenvolvimento de software com produtividade**. v. 1, n. 24, 2020. Apêndice A - Git. Disponível em: <https://link.ufms.br/G5d6h>. Acesso em: 13 jan. 2025.

VALENTE, Marco Tulio. **Engenharia de software moderna: princípios e práticas para desenvolvimento de software com produtividade**. v. 1, n. 24, 2020. Capítulo 10 - DevOps. Disponível em: <https://link.ufms.br/ye4Sd>. Acesso em: 13 jan. 2025.

UNIVERSIDADE FEDERAL DE MATO GROSSO DO SUL (UFMS). **Resolução nº 304-COGRAD/UFMS, de 17 de junho de 2021**. Estabelece as Normas para a curricularização da extensão nos Cursos de Graduação. Disponível em: <https://sei.ufms.br>. Acesso em: 6 mai. 2026.